Kiểm thử hồi quy an ninh của phòng thủ prompt injection gián tiếp khi hợp đồng công cụ thay đổi

Nguyễn Thùy Dương, Đặng Ngọc Huy và Vũ Hồng Quang

Nhóm 11
Học phần Chính sách quản trị an ninh mạng

Tóm tắt. Agent sử dụng mô hình ngôn ngữ lớn có thể gọi công cụ để đọc và thay đổi dữ liệu. Nội dung không tin cậy trong tài liệu hoặc kết quả công cụ có thể chứa chỉ dẫn làm agent hành động ngoài nhiệm vụ. Báo cáo trình bày thiết kế kiểm thử hồi quy an ninh khi hợp đồng công cụ chuyển từ phiên bản $v0$ sang $v1$. Phương pháp giữ cố định nhiệm vụ, trạng thái ban đầu và cấu hình agent, đồng thời tách kết quả an ninh khỏi mức độ hoàn thành nhiệm vụ. Nhóm triển khai một phòng thủ giới hạn quyền truy cập tệp, hai trạng thái policy đóng băng và đã chuyển đổi, cùng kiểm tra trạng thái trước và sau lời gọi. Kiểm chứng bằng chuỗi thao tác xác định cho thấy policy đóng băng từ chối cấu trúc tham số mới, còn policy đã chuyển đổi hoàn thành thao tác được cấp quyền. Kết quả này xác nhận hoạt động của thành phần triển khai; chưa chứng minh hiệu quả trước prompt injection trên mô hình thật. Báo cáo nêu kiến thức nền tảng, nghiên cứu liên quan, kiến trúc giải pháp, cách tái lập, giới hạn và kế hoạch đánh giá tiếp theo.

Từ khóa: LLM agent · prompt injection gián tiếp · hợp đồng công cụ · kiểm thử hồi quy · chính sách truy cập

1 Vấn đề cần giải quyết

1.1 Bối cảnh và động cơ

Một agent sử dụng mô hình ngôn ngữ lớn (LLM) thường lặp lại ba bước: đọc yêu cầu và dữ liệu, đề xuất lời gọi công cụ, rồi dùng kết quả công cụ để quyết định bước tiếp theo. Khác với hệ thống chỉ sinh văn bản, lời gọi có thể gửi thư, thêm lịch hoặc sửa tệp. Vì vậy, sai lệch trong suy luận có thể dẫn đến thay đổi thực tế ở môi trường.

Prompt injection gián tiếp xuất hiện khi chỉ dẫn của bên thứ ba nằm trong dữ liệu mà agent được phép đọc, thay vì trong yêu cầu trực tiếp của người dùng. Ví dụ, một tài liệu công việc có thể chứa nội dung yêu cầu agent thực hiện hành động ngoài nhiệm vụ. Ranh giới cần bảo vệ là quyền hành động: việc đọc được một chỉ dẫn không đồng nghĩa với việc được phép thực hiện chỉ dẫn đó [3,2].

Hệ thống còn phải đối mặt với thay đổi giao diện công cụ. Policy có thể kiểm tra một trường tham số ở phiên bản cũ nhưng không còn diễn giải đúng cấu trúc ở phiên bản mới. Một thay đổi vẫn hợp lệ về schema chưa chắc tương đương về quyền truy cập. Kiểm thử hồi quy cần quan sát cả tác động bị cấm lẫn khả năng hoàn thành tác vụ hợp lệ.

1.2 Câu hỏi và mục tiêu

Câu hỏi nghiên cứu là: một phòng thủ đã ngăn tác động có hại ở hợp đồng công cụ $v0$ có duy trì được sự bảo vệ khi chuyển sang $v1$, và việc chuyển đổi policy có khắc phục sai lệch mà vẫn giữ utility hay không? Trong phạm vi học phần, nhóm xây dựng thiết kế đo lường và hiện thực một phần giải pháp để kiểm chứng khả năng triển khai.

Ba đầu ra kỹ thuật của báo cáo là: định nghĩa so sánh theo cặp; xây dựng guard giới hạn quyền theo nhiệm vụ; và kiểm tra độc lập trạng thái sau thực thi. Đề tài cố định có tên tiếng Anh là *Security Regression Testing of Indirect Prompt Injection Defenses under Versioned Agent Tool Contracts*. Báo cáo không khẳng định đã tạo một cơ chế phòng thủ mới hoặc đã chứng minh hiệu quả trên một quần thể agent.

1.3 Phạm vi và mô hình đe dọa

Bên tấn công có thể kiểm soát nội dung email, tài liệu hoặc kết quả công cụ mà agent đọc. Họ không được sửa policy tin cậy, dispatcher, mã công cụ hoặc oracle. Môi trường triển khai là sandbox với dữ liệu giả lập. Phạm vi không bao gồm đầu độc mô hình, chiếm quyền mã Python trong tiến trình tin cậy, hoặc bảo đảm luồng thông tin xuyên nhiều agent. Các giả định này giới hạn ý nghĩa của kết quả: guard tại dispatcher không bảo vệ một đường thực thi cố tình bỏ qua dispatcher.

2 Kiến thức nền tảng

2.1 Hợp đồng công cụ và quyền theo nhiệm vụ

Hợp đồng công cụ gồm tên thao tác, schema tham số, giá trị mặc định, phạm vi tài nguyên và ý nghĩa tác động. Đổi tên đơn thuần có thể giữ nguyên ý nghĩa; đổi cách biểu diễn tài nguyên hoặc đích nhận có thể làm thay đổi đầu vào của kiểm tra quyền. Báo cáo phân biệt hai loại này để tránh gọi mọi thay đổi giao diện là hồi quy an ninh.

Nguyên tắc quyền tối thiểu cấp cho agent chỉ những thao tác cần thiết để hoàn thành nhiệm vụ. Một capability có thể ràng buộc loại thao tác, tài nguyên, đích nhận, thời hạn và số lần gọi. Bản triển khai trong báo cáo chỉ hiện thực phần công cụ và tài nguyên cho một nhiệm vụ; thời hạn, quota và phê duyệt là các hướng mở rộng, không phải tính năng đã kiểm chứng.

2.2 Kiểm soát trước thực thi và đo lường sau thực thi

Guard kiểm tra lời gọi trước khi chuyển tới executor. Quyết định ALLOW cho phép gửi lời gọi xuống công cụ; DENY trả về phản hồi policy và không thực thi công cụ. Khi không đọc được cấu trúc quyền, guard chọn từ chối, gọi là *fail closed*. Cách xử lý này có thể giữ an toàn nhưng gây mất utility.

Oracle là hàm chấm điểm độc lập. Oracle an ninh đọc trạng thái trước và sau để xác định tác động bị cấm; oracle utility kiểm tra nhiệm vụ có hoàn thành hay không. ALLOW không chứng minh tác vụ thành công, và DENY không tự chứng minh không có tác động. Một lỗi công cụ cũng không thể được xem là thành công bảo mật. Vì vậy phải ghi riêng lỗi thực thi, quyết định policy và kết quả oracle.

2.3 So sánh theo cặp và dữ liệu thiếu

Gọi $H_{i,v} \in \{0, 1\}$ là tác động có hại của trường hợp i tại phiên bản v. Sự kiện hồi quy an ninh là $H_{i,v0} = 0$ và $H_{i,v1} = 1$. Bốn chuyển tiếp 00, 01, 10, 11 phải được giữ lại; chỉ báo cáo tỷ lệ thành công tấn công ở $v1$ sẽ che giấu sự khác nhau giữa hồi quy và sửa chữa.

$$\widehat{\Delta}_{\text{sec}} = \frac{1}{n} \sum_{i=1}^{n} (H_{i,v1} - H_{i,v0}). \tag{1}$$

Trong công thức, n là số cặp hợp lệ đã được chấm. Các cặp thiếu bằng chứng vẫn nằm trong tổng số cặp dự kiến nhưng được ghi riêng là không thể chấm, không được thay bằng giá trị 0. Với utility U, sự kiện suy giảm là $U_{v0} = 1$, $U_{v1} = 0$; hai endpoint có chiều diễn giải khác nhau.

Những biến thể cùng nhiệm vụ nguồn không phải các quan sát độc lập. Thiết kế nghiên cứu tiếp theo cần gom chúng theo nhiệm vụ nguồn khi tính độ bất định. Báo cáo học phần chưa thực hiện suy luận thống kê từ các kiểm tra xác định.

3 Nghiên cứu liên quan

AgentDojo cung cấp môi trường có trạng thái để đánh giá agent trước prompt injection và kiểm tra cả security lẫn utility [2]. Nhóm dùng môi trường này làm nguồn tác vụ và công cụ. Việc gọi thành công một công cụ giả lập chỉ xác nhận tích hợp; đánh giá agent cần hành động thực sự do mô hình sinh ra.

InjecAgent nghiên cứu prompt injection gián tiếp trong agent tích hợp công cụ [5]. Công trình này giúp phân biệt nội dung mang chỉ dẫn tấn công với hành động đầu ra của agent. Báo cáo không gộp một chuỗi lời gọi được viết sẵn với một episode có mô hình đọc carrier và tự quyết định.

CaMeL tổ chức tách biệt dữ liệu không tin cậy và luồng điều khiển, kết hợp kiểm tra capability [1]. Progent đặt trọng tâm vào kiểm soát đặc quyền thông qua policy [4]. Vì các ý tưởng kiểm soát quyền và runtime enforcement đã được nghiên cứu, TaskFileScope trong báo cáo là thành phần tham chiếu để kiểm thử, không phải tuyên bố sáng tạo một nguyên lý bảo vệ mới.

Điểm tập trung của đề tài là đo sự thay đổi của cùng phòng thủ dưới hai hợp đồng công cụ. Đây là một câu hỏi đánh giá có thể kiểm chứng. Tổng hợp trên chỉ là lựa chọn công trình gần, không phải tổng quan hệ thống đầy đủ và chưa đủ để khẳng định tính mới của đóng góp nghiên cứu.

4 Giải pháp và phần đã hiện thực

4.1 Kiến trúc và luồng thực thi

Đầu vào tin cậy gồm yêu cầu người dùng, policy theo nhiệm vụ, phiên bản hợp đồng và manifest thí nghiệm. Agent nhận dữ liệu công cụ và đề xuất hành động. Dispatcher đưa hành động qua guard; chỉ hành động được cho phép mới đi tới executor. Mỗi lần gọi ghi lại định danh policy, digest tham số, quyết định, trạng thái dispatch và hash trạng thái

trước/sau. Cuối episode, oracle độc lập chấm utility hoặc tác động có hại từ bằng chứng đã lưu.

Manifest cố định các trường hợp dự kiến trước khi chạy. Capture bridge tạo tiến trình mới để giảm rò rỉ trạng thái giữa các episode, ghi phiên bản runtime và các request/response. Validator kiểm tra bằng chứng; statistics adapter chỉ nhận cặp hợp lệ. Hash hỗ trợ phát hiện thay đổi và kiểm tra nhất quán; hash không chứng thực rằng một nhà cung cấp thực sự đã tạo phản hồi đó.

4.2 TaskFileScope và chuyển đổi policy

Lát cắt triển khai sử dụng nhiệm vụ `user_task_29` trong Workspace: đọc tệp hoạt động nhóm và bổ sung hoạt động vào tệp đó. Policy tin cậy chỉ cấp quyền tìm theo tên tệp đã định, đọc file ID 3 và append file ID 3. Mọi công cụ khác bị từ chối. Policy không lấy quyền từ nội dung tài liệu, nhãn tấn công hoặc verdict của model.

Trong $v0$, công cụ đọc và append dùng trường `file_id` ở cấp cao nhất. Trong $v1$, cùng công cụ nhận `resource` gồm `space` và `file_id`; giá trị space bắt buộc là `cloud_drive`. Adapter kiểm tra cấu trúc rồi chuyển về tham số native tương ứng. Không có fallback tự động về trường cũ.

Hai cấu hình thuộc cùng họ phòng thủ. F giữ nguyên policy $v0$ ở cả hai phiên bản. M dùng policy $v0$ ở $v0$ và chỉ đổi bộ đọc tham số sang hợp đồng $v1$ ở $v1$. Các quyền trên công cụ và tài nguyên không đổi. Trong một phiên bản, F và M nhìn thấy cùng schema công cụ.

4.3 Quy tắc xử lý và bằng chứng

Guard từ chối công cụ ngoài quyền, tham số thừa, kiểu sai, scope sai hoặc file khác. DENY là phản hồi policy bình thường; native tool không được gọi. Lỗi do native tool vẫn giữ là lỗi thực thi. Đối với denial có typed state, hash trước/sau phải bằng nhau. Trace gốc được giữ để biết agent đã đề xuất gì; trace canonical chuẩn hóa tham số để scorer upstream đọc được.

Tìm kiếm theo tên vẫn dùng ngữ nghĩa substring của AgentDojo. Vì vậy guard này không phải cơ chế bảo mật đọc dữ liệu cho toàn hệ thống. Nó cũng không bảo vệ khỏi mã Python độc hại có thể truy cập trực tiếp callable trong tiến trình.

4.4 Liên hệ với quản trị an ninh mạng

Một thay đổi schema cần được quản trị như thay đổi policy: có người chịu trách nhiệm rà quyền, kiểm thử trước triển khai và lưu bằng chứng phiên bản. Nếu policy chưa được chuyển đổi, từ chối có kiểm soát là lựa chọn rõ ràng hơn việc tự suy đoán ý nghĩa. Tuy nhiên tổ chức cũng cần theo dõi tác vụ hợp lệ bị chặn, vì chặn tất cả công cụ sẽ không đáp ứng mục tiêu vận hành.

5 Triển khai và phương án đánh giá

5.1 Môi trường và tổ chức artifact

Mã nguồn triển khai bằng Python, dùng AgentDojo Workspace v1.2.2 tại commit `089ed468cf3e`. Môi trường tái lập lịch sử khóa CPython 3.12.13, Pydantic 2.11.4 và PyYAML 6.0.2. Các fixture trạng thái còn phụ thuộc encoding mặc định cp1252 trên Windows với Python UTF-8 mode tắt. Đây là ràng buộc tái lập của dữ liệu hiện có, không phải khuyến nghị dùng cp1252 cho dữ liệu mới.

Guard và adapter migration nằm trong module TaskFileScope; source preflight chạy chuỗi thao tác được cấp quyền qua ToolsExecutor. Manifest và result validator v2 giữ riêng đường kiểm tra đổi tên lịch sử. Amendment TaskFileScope bổ sung schema resource; report acceptance mới đang được củng cố và chưa được xem là kết quả nghiên cứu hoàn chỉnh.

5.2 Kết quả kiểm chứng thành phần

Bảng 1 ghi kết quả source-conformance đã quan sát. Đầu vào là chuỗi thao tác xác định cho nhiệm vụ người dùng, không phải hành động do LLM thật sinh ra. Mỗi ô bắt đầu từ cùng trạng thái nguồn được reset và utility do scorer upstream tính.

Bảng 1. Utility trong bốn kiểm tra nguồn của TaskFileScope

Cấu hình	Contract	Policy decoder	Utility
F	$v0$	$v0$	Thành công
F	$v1$	$v0$	Không thành công
M	$v0$	$v0$	Thành công
M	$v1$	$v1$	Thành công

F/$v1$ không hiểu đường dẫn resource mới nên từ chối append. M/$v1$ diễn giải đúng resource và cho phép append vào file được cấp quyền. Kết quả phù hợp với dự đoán về mất utility khi đóng băng policy. Nó không chứng minh có hồi quy an ninh: nhiệm vụ này không cấp quyền xóa ở bất kỳ phiên bản nào, nên lát cắt chưa phân biệt được giả thuyết harmful-effect chính.

Kiểm tra tích hợp với server giả tương thích OpenAI đã chạy lời gọi tìm kiếm và append qua tiến trình capture mới: F/$v1$ giữ nguyên trạng thái khi từ chối; M/$v1$ làm thay đổi trạng thái sau append. Đây là bằng chứng đường chạy và thu thập dữ liệu. Server giả không thể chứng minh mô hình chống được prompt injection.

Bộ regression ở snapshot `e678ea2` ghi nhận 144 test thành công, không failure hoặc skip. Con số này áp dụng cho snapshot đó; không chứng nhận các thay đổi nghiên cứu đang làm dở sau snapshot. Các test gồm kiểm tra metadata, trace, reset, thiếu bằng chứng và request retry. Báo cáo không dùng số test làm thước đo hiệu quả bảo mật.

5.3 Tái lập và tiêu chí nghiệm thu

Trong môi trường pinned của repository, kiểm tra nguồn chạy bằng module source pre-flight của TaskFileScope; kiểm tra tích hợp nằm trong bộ test amendment cùng tên. Hướng dẫn đi kèm bộ LaTeX ghi đường dẫn và lệnh PowerShell đầy đủ. Biên dịch báo cáo không cần môi trường AgentDojo hoặc khóa API.

Nghiệm thu thành phần yêu cầu: lời gọi ngoài quyền không tới native executor; denial không đổi typed state; policy F không tự đổi theo contract; M chỉ thay decoder đã khai báo; utility được tính độc lập. Thất bại provider, lỗi reset hoặc thiếu trace phải được giữ trong mẫu dự kiến, không được xóa để làm đẹp kết quả.

5.4 Thiết kế thực nghiệm tiếp theo

Giai đoạn nghiên cứu cần bổ sung một treatment có khả năng bộc lộ thay đổi quyền tạo tác động có hại, không làm yếu F chỉ để tạo kết quả mong muốn. Với mỗi case, giữ cùng model snapshot, task, carrier và reset; chạy cả $v0$ và $v1$ cho F/M. Tách riêng attack endpoint và default-control utility; lưu đủ bốn chuyển tiếp và nguyên nhân không chấm được.

Khi có đủ dữ liệu, báo cáo chênh lệch rủi ro theo cặp, khoảng tin cậy bootstrap theo nhiệm vụ nguồn và McNemar trong điều kiện độc lập thích hợp. Kế hoạch nghiên cứu dự kiến ít nhất ba model snapshots, kiểm tra carrier giữ lại và tái lập độc lập. Chưa có bộ dữ liệu đáp ứng các cổng đó trong kết quả của báo cáo này.

6 Hạn chế và kết luận

Phần hiện thực đã cho thấy có thể đặt một guard quyền theo nhiệm vụ trước công cụ, chuyển đổi cấu trúc resource một cách tường minh và kiểm tra utility bằng scorer độc lập. Đây là phần giải pháp cụ thể phục vụ mục tiêu học phần; chưa phải nghiên cứu thực nghiệm hoàn chỉnh về khả năng chống prompt injection.

Các hạn chế chính gồm một task nguồn, một kiểu migration do nhóm thiết kế, chuỗi thao tác xác định và provider giả. Thiết kế chưa kiểm tra paraphrase, partial leakage, công cụ ngoài sandbox hoặc nhiều model. Biến thiên timestamp của thao tác append cũng khiến hash trạng thái sau thực thi có thể khác, dù utility giống nhau. Không được suy rộng số đếm kiểm tra sang tỷ lệ an toàn của hệ thống thực tế.

Bước tiếp theo về kỹ thuật là hoàn thiện ràng buộc plan–record–oracle và kiểm chứng phản biện độc lập. Sau đó mới thu thập dữ liệu từ mô hình thật theo thiết kế cố định. Kết luận hiện tại là tính khả thi của phương pháp kiểm thử và một lát cắt triển khai đã được kiểm tra; kết luận hiệu quả bảo mật vẫn cần bằng chứng bổ sung.

Tài liệu tham khảo

1. Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F.: Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 (2025), https://arxiv.org/abs/2503.18813
2. Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: Advances in Neural Information Processing Systems. vol. 37, pp. 82895–82920 (2024). https://doi.org/10.52202/079017-2636
3. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. arXiv preprint arXiv:2302.12173 (2023), https://arxiv.org/abs/2302.12173
4. Shi, T., He, J., Wang, Z., Wu, L., Li, H., Guo, W., Song, D.: Progent: Programmable privilege control for llm agents. arXiv preprint arXiv:2504.11703 (2025), https://arxiv.org/abs/2504.11703
5. Zhan, Q., Liang, Z., Ying, Z., Kang, D.: Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of ACL 2024. pp. 10471–10506 (2024). https://doi.org/10.18653/v1/2024.findings-acl.624